

Introduction to the Theory of Computation

Lecture 1. What This Course Is About — A Guided Tour

Neo (Yuan) Li

July, 2026

Fudan University

yuan_li@fudan.edu.cn

Computation is everywhere.



Figure 1: Mosquitos

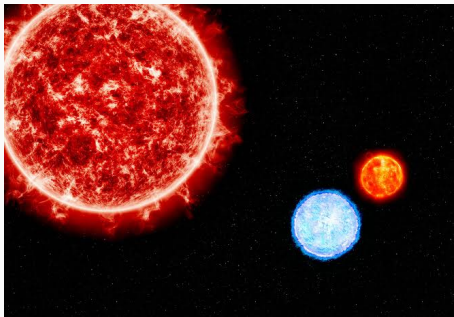


Figure 2: 3-body problem



Figure 3: Human brain

Computation is everywhere.



Figure 4: Desktop computer

Computation is everywhere.



Figure 5: Datacenter

Computation is everywhere.



Figure 6: Internet

Computation is everywhere.



Figure 7: Quantum computer

- (Models) What is computation?
- (Computability) What can be computed?
- (Complexity) What can be computed efficiently?

Models of Computation

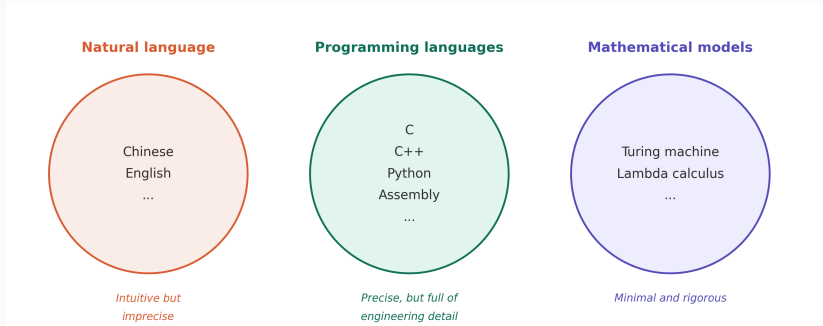


Figure 8: 3 ways to define computation

Example: Euclid's GCD

In natural language (English):

To find the greatest common divisor of two positive integers a and b : repeatedly replace the larger number by its remainder when divided by the smaller number, until one of the numbers becomes zero. The other number is then the greatest common divisor.

In C:

```
int gcd(int a, int b) {  
    while (b != 0) {  
        int r = a % b;  
        a = b;  
        b = r;  
    }  
    return a;  
}
```

Models of Computation

```
177 static int alx_set_link_ksettings(struct net_device *netdev,  
178                                 const struct ethtool_link_ksettings *cmd)  
179 {  
180     struct alx_priv *alx = netdev_priv(netdev);  
181     struct alx_hw *hw = &alx->hw;  
182     u32 adv_cfg;  
183     u32 advertising;  
184  
185     ASSERT_RTNL();  
186  
187     ethtool_convert_link_mode_to_legacy_u32(&advertising,  
188                                             cmd->link_modes.advertising);  
189  
190     if (cmd->base.autoneg == AUTONEG_ENABLE) {  
191         if (advertising & ~alx_get_supported_speeds(hw))  
192             return -EINVAL;  
193         adv_cfg = advertising | ADVERTISED_Autoneg;  
194     } else {  
195         adv_cfg = alx_speed_to_ethadv(cmd->base.speed,  
196                                       cmd->base.duplex);  
197  
198         if (!adv_cfg || adv_cfg == ADVERTISED_1000baseT_Full)  
199             return -EINVAL;  
200     }  
201  
202     hw->adv_cfg = adv_cfg;  
203     return alx_setup_speed_duplex(hw, adv_cfg, hw->flowctrl);  
204 }  
205
```

Figure 9: C program

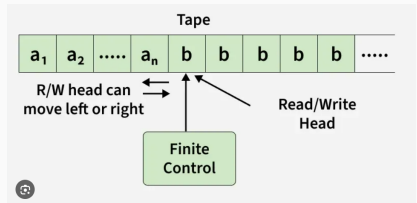


Figure 10: Turing machine

Mathematically, a Turing machine is a 6-tuple

$$(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{end}}),$$

where

1. Q is the set of states,
2. Σ is the input alphabet and Γ is the tape alphabet,
3. $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ is the transition function,
4. $q_0 \in Q$ is the start state,
5. $q_{\text{end}} \in Q$ is the end state.

Turing's 1936 paper

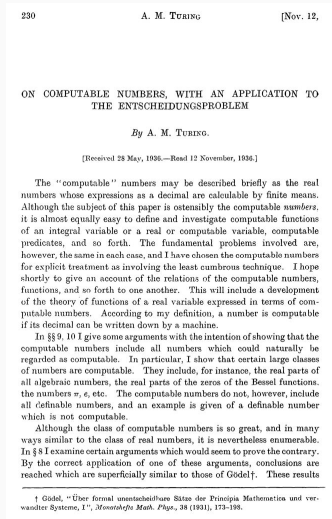


Figure 11: Turing's 1936 paper

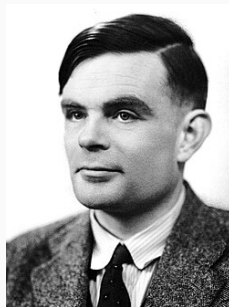


Figure 12: Alan Turing (1912 – 1954)

A landmark paper: Turing's 1936 "On Computable Numbers, with an Application to the Entscheidungsproblem," which laid the foundation for the modern theory of computation.

Not all problems can be solved by an algorithm
— not all problems are **computable**.

Turing Halting Problem

There is no algorithm that can determine, for every program and input, whether the program will eventually halt.

Computable vs Uncomputable problems

Computable

Sorting a list
GCD (Euclid's algorithm)
Winning strategy in Go
Checking if a number is prime
...

Uncomputable

The Halting Problem
Will this program crash?
Hilbert's 10th Problem
Word Problem for Groups
...

Hilbert's Tenth Problem

Does a given Diophantine equation have an integer solution?

Example 1 (has a solution):

$$x^2 - 2y^2 = 1 \quad \text{— yes, e.g. } (x, y) = (3, 2)$$

Example 2 (no solution):

$$x^2 + y^2 = 3 \quad \text{— no integer solution}$$

(since squares mod 4 are only 0 or 1, their sum mod 4 can only be 0, 1, 2 — never 3)

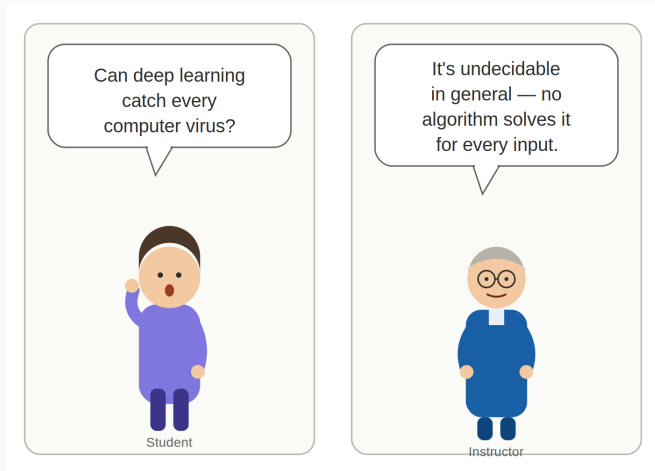
Word Problem for Groups

Given a finitely presented group and a word, is it equivalent to the identity?

e.g. $G = \langle a, b \mid a^2 = b^3 = (ab)^5 = 1 \rangle \cong A_5$, $w = aba^{-1}b^{-1}$

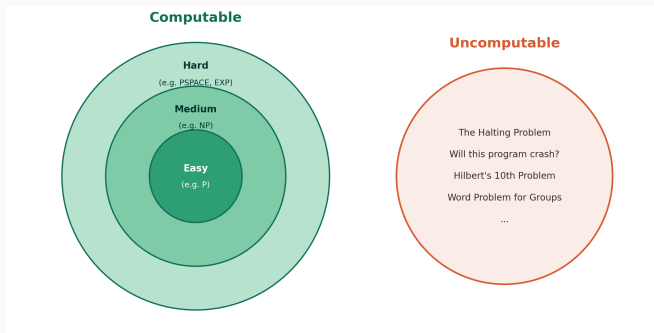
— no: if $w = e$ then a, b commute, so $\langle a, b \rangle$ is abelian; but $\langle a, b \rangle = A_5$ is non-abelian, contradiction.

A practical example: virus detection



F. Cohen, "Computer Viruses: Theory and Experiments," *Computers & Security*, 1987.

Complexity theory studies the inherent resources — time, space, energy, etc. — a problem requires, categorizing problems by their difficulty.



Complexity classes

- **NC** — solvable efficiently *in parallel* (polylog time, poly processors)
- **P** — solvable by a polynomial-time algorithm
- **NP** — a candidate solution can be *verified* in polynomial time
- **PSPACE** — solvable using only polynomial *space* (time can be exponential)

$$\text{NC} \subseteq \text{P} \subseteq \text{NP} \subseteq \text{PSPACE}$$

P vs NP problem

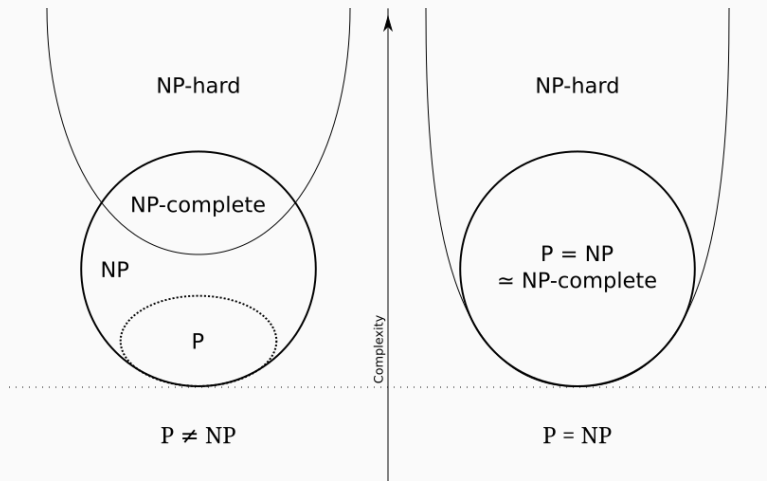


Figure 13: P vs NP

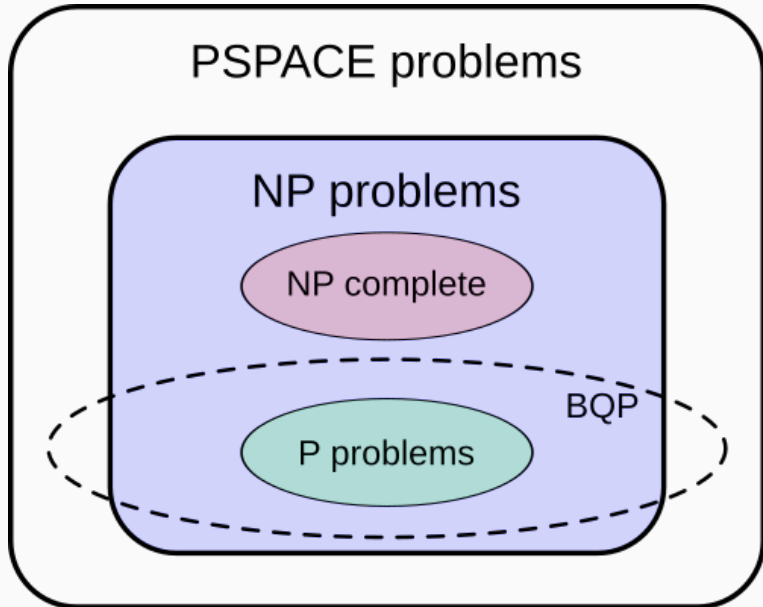
NP-complete problems: Solving any one efficiently solves all of NP.

Takeaway: Stop chasing optimal for NP-hard problems — go approximate.

Many real-world problems fall outside the scope of complexity theory entirely.

- Face recognition
- Drug / molecule design
- Text-to-image generation

Their complexity is **unknown** — because these problems cannot be precisely defined mathematically.



The Theory of Computation is a foundational pillar, connected to many other fields:

- **Mathematics** — logic, proof systems, decidability
- **Programming languages & compilers** — grammars, parsing, semantics
- **Cryptography** — hardness assumptions, zero-knowledge proofs, blockchain
- **AI / machine learning** — learnability, complexity, and the limits of computation
- **Physics** — quantum computing, simulation of physical systems

How secure is a Bitcoin private key?

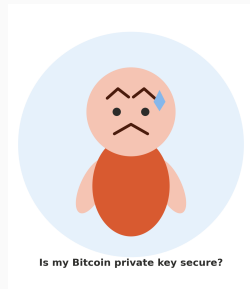
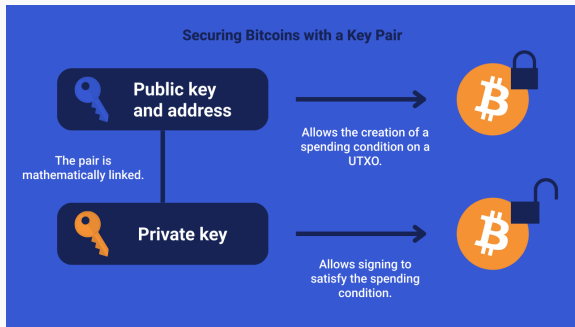


Figure 15: Bitcoin keys